

COMPUTER SCIENCE TRIPOS Part IB – 2021 – Paper 4

8 Semantics of Programming Languages (nk480)

Languages like FORTH and POSTSCRIPT are *stack-based languages*; they store intermediate values on a stack rather than binding to variable names. In this question we will look at how to give a type system and operational semantics for a simple stack-based language. The syntax and informal meaning of our language is given by:

|              |                                    |                                                                                                                                |
|--------------|------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| $e ::=$      | $\underline{n}$                    | Push the numeral $n$ on the stack                                                                                              |
|              | $\underline{b}$                    | Push the Boolean $b$ on the stack                                                                                              |
|              | <b>Add</b>                         | Replace the top two stack elements with their sum                                                                              |
|              | <b>Eql</b>                         | Replace the top two stack elements with the result of comparing them for equality                                              |
|              | <b>Cond</b> ( $e_1, e_2$ )         | Delete the top stack element and execute $e_1$ or $e_2$ , depending on if the top of the stack was <b>True</b> or <b>False</b> |
|              | <b>Skip</b>                        | No-op                                                                                                                          |
|              | $e_1; e_2$                         | Run $e_1$ and then $e_2$                                                                                                       |
| $v ::=$      | $\underline{b} \mid \underline{n}$ | Values                                                                                                                         |
| $s ::=$      | $\cdot \mid s, v$                  | Stacks                                                                                                                         |
| $\tau ::=$   | <b>bool</b> $\mid$ <b>num</b>      | Types                                                                                                                          |
| $\Gamma ::=$ | $\cdot \mid \Gamma, \tau$          | Stack Types                                                                                                                    |

We take a value  $v$  to be a Boolean or numeral, and define a stack  $s$  to be a stack of values (growing at the right). Correspondingly, there are types **bool** and **num** for values, and stack types  $\Gamma$  for stacks  $s$ .

The small-step operational semantics is then defined by a transition relation  $\langle e_1 \mid s_1 \rangle \mapsto \langle e_2 \mid s_2 \rangle$ . One rule for this relation is:

$$\overline{\langle \text{Add} \mid s, \underline{n}, \underline{m} \rangle \mapsto \langle \text{Skip} \mid s, \underline{n + m} \rangle}$$

The typing relation is given as a relation  $\Gamma \vdash e \dashv \Gamma'$ , which means that  $e$ , when run with a stack of shape  $\Gamma$ , yields a stack of shape  $\Gamma'$ . One rule for this relation is:

$$\overline{\Gamma, \text{num}, \text{num} \vdash \text{Add} \dashv \Gamma, \text{num}}$$

(a) Give the remaining rules for the operational semantics. [7 marks]

---

*Answer:*

$$\begin{array}{ll} \overline{\langle \underline{b} \mid s \rangle \mapsto \langle \text{Skip} \mid s, \underline{b} \rangle} & \overline{\langle \underline{n} \mid s \rangle \mapsto \langle \text{Skip} \mid s, \underline{n} \rangle} \\ \overline{\langle \text{Eql} \mid s, \underline{n}, \underline{m} \rangle \mapsto \langle \text{Skip} \mid s, \underline{n = m} \rangle} & \\ \overline{\langle \text{Cond}(e_1, e_2) \mid s, \underline{t} \rangle \mapsto \langle e_1 \mid s \rangle} & \overline{\langle \text{Cond}(e_1, e_2) \mid s, \underline{f} \rangle \mapsto \langle e_2 \mid s \rangle} \\ \overline{\langle e_1 \mid s \rangle \mapsto \langle e'_1 \mid s' \rangle} & \\ \overline{\langle e_1; e_2 \mid s \rangle \mapsto \langle e'_1; e_2 \mid s' \rangle} & \overline{\langle \text{Skip}; e_2 \mid s \rangle \mapsto \langle e_2 \mid s \rangle} \end{array}$$

- 
- (b) Give the remaining rules for the typing judgement. [7 marks]
- 

*Answer:*

$$\begin{array}{c}
 \overline{\Gamma \vdash \underline{b} \dashv \Gamma, \text{bool}} \qquad \overline{\Gamma \vdash \underline{n} \dashv \Gamma, \text{num}} \\
 \\
 \overline{\Gamma, \text{num}, \text{num} \vdash \text{Eq} \dashv \Gamma, \text{bool}} \\
 \frac{\Gamma \vdash e_1 \dashv \Gamma' \quad \Gamma \vdash e_2 \dashv \Gamma'}{\Gamma, \text{bool} \vdash \text{Cond}(e_1, e_2) \dashv \Gamma'} \quad \overline{\Gamma \vdash \text{Skip} \dashv \Gamma} \\
 \frac{\Gamma \vdash e_1 \dashv \Gamma' \quad \Gamma' \vdash e_2 \dashv \Gamma''}{\Gamma \vdash e_1; e_2 \dashv \Gamma''}
 \end{array}$$


---

- (c) Formulate and state the progress and preservation lemmas for this language. [6 marks]
- 

*Answer:* To solve this question, students will first have to define a stack typing relation  $s : \Gamma$ . With that in hand, they can adapt progress and preservation theorems to this language:

- (i) (Progress) If  $s : \Gamma$  and  $\Gamma \vdash e \dashv \Gamma'$  then either  $e = \text{Skip}$  or there are  $e'$  and  $s'$  such that  $\langle e \mid s \rangle \mapsto \langle e' \mid s' \rangle$ .
- (ii) (Preservation) If  $s : \Gamma$  and  $\Gamma \vdash e \dashv \Gamma''$  and  $\langle e \mid s \rangle \mapsto \langle e' \mid s' \rangle$ , then there is a  $\Gamma'$  such that  $s' : \Gamma'$  and  $\Gamma' \vdash e' \dashv \Gamma''$ .
-